

# PowerGPU: Power-aware Feedback Control GPU Scheduling in Real-time Embedded Systems

Srinivasan Subramaniyan, Blake Morse, and Xiaorui Wang

The Ohio State University, Columbus, Ohio, USA

{subramaniyan.4, morse.216, wang.3596}@osu.edu

**Abstract**—Traditional GPU scheduling heavily relies on off-line schedulability analysis and thus cannot adapt to runtime workload changes. Recently proposed adaptive solutions, such as Response Time Ratio (RTR) control, have been shown to be effective for meeting deadlines in GPU-based real-time embedded systems operating in open or unpredictable environments. However, current research on response time control focuses exclusively on task-rate adaptation, which cannot effectively handle tasks whose rates are fixed or can be adjusted only within small ranges.

In this paper, we present *PowerGPU*, a two-tier coordinated RTR control architecture for GPU-based real-time embedded systems. The outer control loop uses frequency scaling to control the average RTR of all tasks, while the inner control loop uses rate adaptation to control the individual RTR of each task on a finer timescale. Empirical results on a physical GPU testbed show that our control solution outperforms a state-of-the-art RTR control algorithm, achieving more precise control and having 29.4% lower power consumption.

**Index Terms**—Real-time systems, GPU scheduling, response time ratio control, rate adaptation, and feedback control

## I. INTRODUCTION

GPUs have been widely used in real-time embedded applications [1]–[4], such as virtual reality for video rendering, drones for object detection, and autonomous vehicles for path planning. Existing scheduling solutions for managing real-time tasks on GPUs depend on schedulability analysis, which necessitates prior knowledge of the tasks' Worst-Case Execution Times (WCET) to ensure there are no deadline misses in an *open-loop* manner. While these solutions work effectively in closed execution environments, they may violate the desired timing constraints or significantly underutilize the system when task execution times vary dynamically. Recently, feedback control techniques have demonstrated great potential for providing real-time guarantees by adapting to workload variations through dynamic feedback. In particular, feedback-based GPU response time ratio (RTR) control [3] has been demonstrated to be an effective approach for meeting deadlines in soft real-time systems using embedded GPUs.

However, previous research (e.g., [3]) on GPU RTR control relies exclusively on task-rate adaptation, assuming that the invocation rates of periodic tasks can be continuously tuned within wide ranges. While rate adaptation is an effective actuator, it has several limitations. First, it is often infeasible to achieve the desired RTR set points solely through rate

adaptation [1], [5]. For example, many embedded systems are configured based on tasks' WCETs. Consequently, even when all tasks are running at their maximum rates, the RTR can remain well below the desired set points, leading to severely underutilized systems and excessive power consumption. In this case, GPU frequency scaling can be used to reduce power consumption while keeping the *response times* slightly below the deadlines. Second, the model of task rate in many applications can be complex and may vary at runtime as the application evolves [6]–[8]. As a result, the estimated task rate ranges are often inaccurate and may vary significantly online, leading to unexpected rate saturation and even deadline misses when RTR exceeds the set points and cannot be lowered solely through rate adaptation.

This paper presents *PowerGPU*, a two-tier coordinated GPU RTR control architecture. The *outer loop* uses frequency scaling to control the average RTR of all tasks. In the meantime, the *inner loop* uses rate adaptation to control the RTR of each individual task on a finer timescale. The advantages of our solution are two-fold: 1) When RTR is higher than the set point (i.e., response time longer than period, which is also the deadline) and when the task rates are saturated, *PowerGPU* meets task deadlines by using a higher GPU frequency to shorten response times. 2) When the RTR is lower than the set point, *PowerGPU* saves average power of the GPU.

## II. PROBLEM FORMULATION

We introduce the task model and formulate the problem.

### A. Task Model

In real-time systems, a task is implemented as a process or thread that must be scheduled to execute on the target GPU at particular time intervals. Each task has multiple instances, known as jobs. The release time of a job is the time at which the job becomes ready to be scheduled and run on the hardware. To simplify the design, we assume that all tasks are *periodic*. We assume that each task's deadline equals its period. We now introduce the notations used throughout the paper. A system consists of  $N$  tasks ( $t_i$ ,  $1 \leq i \leq N$ ), where each task  $t_i$  is defined by the following parameters.

- $r_i(k)$ : Task rate of  $t_i$  in the  $k^{th}$  control period.
- $e_i$  = Estimated execution time of  $t_i$  on the GPU.
- $p_i(k) = 1/r_i(k)$ : Period of  $t_i$  in the  $k^{th}$  control period.
- $d_i(k)$ : Deadline of  $t_i$  (assumed  $d_i(k) = p_i(k)$ ).

- $f(k)$ : Relative GPU frequency in the  $k^{th}$  control period (i.e., GPU frequency normalized to its highest level).
- $q_i(k)$ : Response time of  $t_i$  in the  $k^{th}$  control period.
- $s$ : RTR set point for all tasks.
- $rtr_i(k)$ : Response Time Ratio (RTR) of  $t_i$ .
- $\theta_i$ : Context switching overhead of  $t_i$

This task model is applicable to a wide range of periodic tasks. We represent each periodic task as  $t(e_i, p_i) \in \mathbb{R}_+^2$ , where the execution time  $e_i$  and the period  $p_i$  are expressed in milliseconds.

### B. Problem Formulation

The optimization formulation minimizes the difference between the RTR of each task and its corresponding set point by manipulating the rate of each task and the GPU frequency within their constraints. Let  $\mathcal{C}$  denote the set of tasks with adjustable rates and  $\mathcal{F}$  the set of tasks with fixed rates. The optimization problem is defined as  $\min_{r_i(k), f(k)} \sum_{i=1}^N (s - rtr_i(k))^2$ . The problem must be constrained to ensure three things: rates for each controllable task  $i$  in  $\mathcal{C}$  must remain within its boundaries,  $[r_{i \min}, r_{i \max}]$ . Second, these controllable tasks can adjust their rates  $r_i(k)$ , while non-controllable tasks in  $\mathcal{F}$  must maintain fixed rates determined by the application. Finally, frequency scaling is restricted to the range  $[f_{\min}, f_{\max}]$ .

## III. SYSTEM ARCHITECTURE

In Section III-A, we present the system model, while in Section III-B, we present the overall architecture of PowerGPU.

### A. System Modeling

As observed in previous research [3], when tasks are launched from separate processes, the response time of one task is significantly affected by the rates of co-scheduled tasks. As well, previous research has shown that the task execution time on GPUs scales inversely with frequency [9]. Since GPUs used in edge servers (RTX 3090) and embedded systems share a single voltage rail [10], the GPUs have a single voltage supply line, and the clock frequency across all SMs is the same. As a result, the response time of a task is modeled as shown below for a system with  $N$  tasks and GPU core frequency  $f(k)$ :

$$q_i(k) = \frac{e_i + \sum_{j=1, j \neq i}^N (e_j + \theta_j) \cdot r_j(k)}{f(k)}, \quad \forall i = 1, \dots, N. \quad (1)$$

As discussed in Section II, the RTR of a task is its ratio of response time to period, meaning we can multiply both sides of Equation (1) by  $r_i(k)$  to get

$$rtr_i(k) = \frac{e_i \cdot r_i(k) + \sum_{j=1, j \neq i}^N (e_j + \theta_j) \cdot r_j(k) \cdot r_i(k)}{f(k)} \quad \forall i = 1, \dots, N. \quad (2)$$

To facilitate controller design later, we derive a difference equation based on the model in Equation (2) using the steps

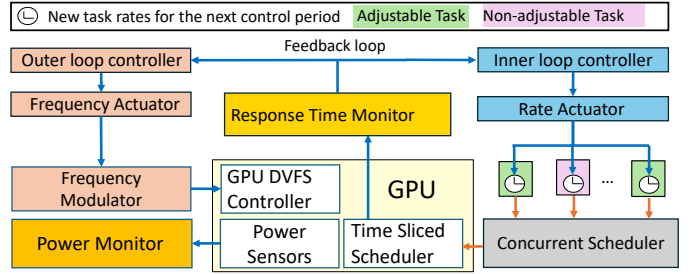


Fig. 1: Overall framework of PowerGPU. The framework consists of a two-tier controller. The control loop periodically monitors each task's response time ratio (RTR); The *outer loop* uses *frequency scaling* to control the average RTR of all tasks. In the meantime, the *inner loop* uses *rate adaptation* to control the RTR of each individual task on a finer timescale.

below. We can use Equation (2) to help define the expected change in RTR for control period  $k$ ,  $\Delta \hat{rtr}_i(k)$ :

$$\Delta \hat{rtr}_i(k) = rtr_i(k) - rtr_i(k-1). \quad (3)$$

Note that  $\Delta \hat{rtr}_i(k)$  is computed using the estimated workload parameters ( $e_j + \theta_j$ ). Since the actual RTR deviates from this estimate due to workload variations and runtime effects, we model the measured RTR using the following difference equation:  $rtr_i(k) = rtr_i(k-1) + g_i \cdot \Delta \hat{rtr}_i(k)$ , where the gain  $g_i$  captures the mismatch between the estimated and observed response-time dynamics. The exact value of  $g_i$  is unknown at design time due to the inherent unpredictability of task execution times and runtime interference effects.

Unfortunately, the system model in Equation (III-A) is nonlinear for two reasons: first,  $\Delta \hat{rtr}_i(k)$  is dependent on multiple inputs (task rate  $r_i(k)$  and frequency  $f(k)$ ) and second,  $rtr_i(k)$  is nonlinear with respect to frequency  $f(k)$ , as seen in Equation (2). To reduce controller complexity and ensure acceptable runtime overhead, linearization is required for controller design. For the inner control loop, we assume that the GPU operates at a static frequency and the RTR is regulated solely through *rate adaptation*. Under this assumption,  $f(k) = 1$  and Equation (2) reduces to a linear relationship between RTR  $rtr_i(k)$  and task rate  $r_j(k)$ . For the second loop, we assume that the RTR is controlled exclusively by frequency scaling, with task rates remaining constant. In this case,  $r_i(k)$  is fixed, and the system model becomes linear with respect to the frequency-related term  $\Delta c(k) = \frac{1}{f(k)} - \frac{1}{f(k-1)}$ .

### B. Overall Architecture

In this paper, we propose a two-tier RTR control architecture, as shown in Figure 1. To avoid a nonlinear model, our control architecture features two coordinated control loops operating on different control periods.

First, the task-level *rate adaptation* loop dynamically controls the RTR across all tasks by adjusting task rates within their allowed ranges. Because the rate change of a task affects the RTR of all the other tasks and this loop is a MIMO control loop, which works as follows: (1) the response time monitor

records the response time of the task  $q_i(k)$  in the last control period and forwards the measurement to the MIMO controller; (2) the controller computes a new rate  $r_i(k)$  for every task  $t_i$  and sends the new rates to the rate modulators; and (3) the rate modulators change the task rates accordingly. In scenarios where a task only supports discrete rate increments, the rate modulator will not round to the nearest value. Instead, it automatically truncates the calculated continuous rate to the next available discrete threshold.

Second, the GPU-level controller regulates the RTR by scaling the GPU's frequency. The controller is a Single-Input Single-Output (SISO) controller because changes in the GPU frequency affect the response times of all tasks running on the GPU. This loop works as follows: (1) the response time monitor sends the response time error to the SISO controller; (2) the controller computes the frequency adjustment  $\Delta f_i(k)$  based on the control error; (3) the actuator forwards the new operating frequency to the frequency modulator, which adjusts the hardware frequency accordingly.

For the rate adaptation controller, we adopt the design in FC-GPU [3] with minor modifications, which is also used as our baseline for a fair comparison. Therefore, due to space limitations, we do not repeat its design here. The DFS controller is our main contribution and is described in detail in the next section.

#### IV. GPU FREQUENCY SCALING LOOP

In this section, we model, design, and analyze the controller for the GPU frequency scaling loop.

##### A. System Model

Based on the two-tier control architecture described in Section III, the GPU *frequency scaling* loop operates on a coarser timescale and can be designed independently of the *rate adaptation* loop. Accordingly, for the design of the *frequency scaling* controller, the task rates  $r_i(k)$  and  $r_j(k)$  in Equation (2) can be assumed to be constants ( $r_i$  and  $r_j$ ). This decouples the *non-linear* control problem because, as discussed in Section III-A, the tasks are coupled to each other due to the fact that the rate change of a task may affect the RTR of all the other tasks. However, assuming the task rates as a constant would control the RTR through frequency adaptation.

Under the above assumption, Equation (3) can be rewritten as shown below.

$$\begin{aligned} \Delta r \hat{r}_i(k) &= rtr_i(k) - rtr_i(k-1) \\ &= \left( e_i r_i + \sum_{j \neq i}^N (e_j + \theta_j) r_j r_i \right) \left( \frac{1}{f(k)} - \frac{1}{f(k-1)} \right), \end{aligned} \quad (4)$$

where  $\frac{1}{f(k)} - \frac{1}{f(k-1)}$  denotes the change in the inverse GPU frequency between two consecutive control periods.

Then, the RTR of task  $t_i$  can be approximated by the following difference equation:

$$rtr_i(k) = rtr_i(k-1) + gh \cdot \Delta c(k-1), \quad (5)$$

where  $h = e_i r_i + \sum_{j \neq i}^N (e_j + \theta_j) r_j r_i$ . The model cannot be used directly to design a controller because the system gain is required to model uncertainties in task execution times and is thus unknown at design time. Therefore, we design the controller based on an approximate system as shown in Equation (5) with  $g = 1$ .

We use a simple, low-overhead Proportional (P) controller for the GPU Frequency scaling loop. More details on the SISO controller design can be found in [11]. Although changes in GPU frequency affect the response times of all tasks, the controller uses a single control input (GPU frequency) and a single aggregated feedback signal.

#### V. SYSTEM IMPLEMENTATION

**Hardware testbed.** We use an Nvidia GeForce RTX 3090 as our primary experimental testbed, because it has the same Ampere architecture that is used in popular autonomous driving systems (e.g., the Nvidia DRIVE Orin and Thor).

**Baselines.** We compare our proposed approach with FC-GPU [3].

**Workloads.** We select workloads from Nvidia Samples and the Rodinia Benchmark for testing.

#### VI. EVALUATION RESULTS

We first show the limitations of the baseline FC-GPU and how PowerGPU performs better with a joint DFS loop.

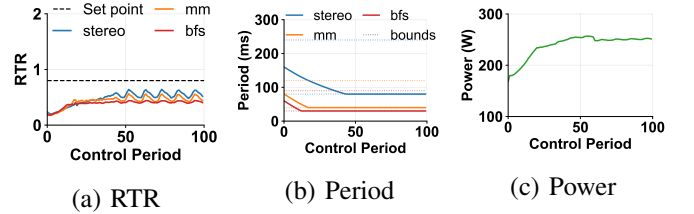


Fig. 2: Limitations of *rate adaptation* only. (a) RTR values for all tasks remain below the set points. (b) Task rates are saturated at the lowest bounds. (c) High power consumption.

##### A. Limitation of Rate Adaptation Only

In this experiment, we demonstrate a scenario in soft real-time systems in which task rates are configured ahead of time, ensuring no deadline misses, as in prior work. Here, the GPU is configured to operate at its highest frequency to ensure no deadline misses. We choose three periodic tasks: *stereo* (23,60), *mm* (12,70), *bfs* (49,80).

As shown in Figure 2(a), the RTR values are significantly lower than the specified set points. When FC-GPU attempts to reduce the task rates, it saturates at the lowest task bounds as shown in Figure 2(b); consequently, the RTR remains significantly below the set point. However, because the GPU operates at its maximum frequency, FC-GPU incurs high power consumption (Figure 2(c)), which is the highest among all the solutions unnecessarily, as shown later in Figure 5(b). More importantly, if RTR exceeds the set points, FC-GPU would increase task rates but can saturate at the upper bounds, causing deadline misses. Hence, relying solely on rate adaptation is insufficient for real-time systems.

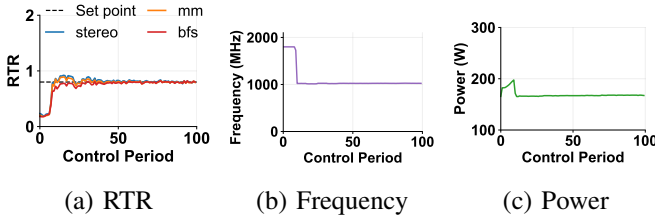


Fig. 3: Effectiveness of PowerGPU. (a) RTR values for all tasks are maintained close to the desired set points. (b) GPU frequency is jointly adapted. (c) Reduced power consumption as the GPU operates at lower frequencies.

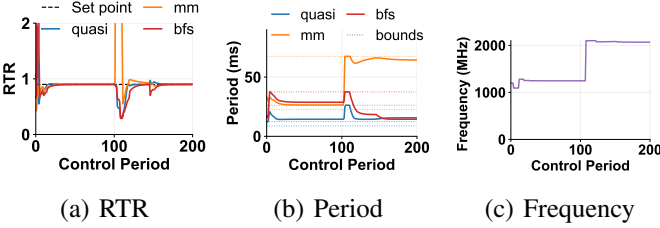


Fig. 4: PowerGPU handles runtime execution time increase. (a) RTR is regulated to remain close to the desired set point, even under workload increase. (b) Task rates and (c) GPU Frequency are jointly adapted to control RTR.

#### B. PowerGPU: Two-Tier Solution

We now test PowerGPU in the same scenario to demonstrate that it can ensure tasks meet their RTR set points while reducing GPU power consumption. We reuse the task configuration described in Section VI-A. Unlike the baselines, PowerGPU combines both *rate adaptation* and *GPU frequency scaling* to regulate system performance.

As shown in Figure 3(a), the RTR of all tasks consistently meets the desired set point. PowerGPU adapts GPU frequency jointly with rate adaptation to ensure no deadline misses, as shown in Figures 3(b). Figure 3(c) illustrates the total power consumption over time. The average power consumption of PowerGPU is 168 W. In contrast, FC-GPU exhibits an average power consumption of 238 W, as shown in Figure 2(c). Overall, PowerGPU achieves substantial power savings, reducing the average power consumption by approximately 29.4% compared to FC-GPU, while maintaining desired RTR.

Next, we stress test PowerGPU in a scenario where the task execution time varies suddenly (e.g., due to detected objects) in the middle of the run, causing deadline misses. As shown in Figure 4(a), at control period 100, there is a noticeable increase in RTR across all tasks due to a sudden increase in task execution time. However, PowerGPU leverages *rate adaptation* and *GPU frequency scaling* to quickly adjust task rates and GPU frequency within the permitted bounds as illustrated in Figures 4(b) and 4(c) and ensuring that the RTR of all tasks converges back to their set points (Figure 4(a)).

#### C. Control Accuracy under Different Set Points

In this section, we evaluate the performance of PowerGPU across a range of RTR set points.

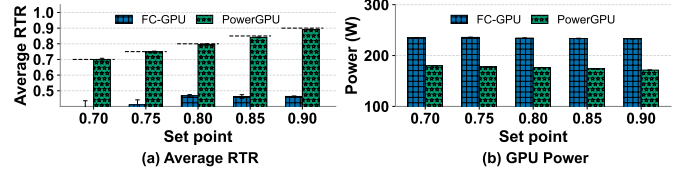


Fig. 5: Average power consumption of all solutions across different set points on the RTX 3090 GPU.

We evaluate three tasks with the following configurations: *hist* (31, 120), *quasi* (4, 35), and *mm* (7, 120). We evaluate the experiments for RTR set points ranging from 0.70 to 0.90 in increments of 0.05. Figure 5(a) illustrates the variation in RTR for all solutions across the considered set points. FC-GPU cannot drive all tasks to the desired set point because task rates saturate before the target is reached. For PowerGPU, Figure 5(a) shows that all tasks converge closely to their respective set points, as indicated by the mean RTR values consistently matching the targets across all benchmarks.

We next compare the power consumption of all solutions. As shown in Figure 5(b), FC-GPU exhibits the highest power consumption. This is because it executes the tasks at the highest rates, thereby maintaining the GPU in a high-power operating region. On average, the proposed solution reduces power consumption by 24.7% compared to FC-GPU.

## VII. CONCLUSION

In this paper, we have formulated a new control problem based on both frequency scaling and rate adaptation. Since a centralized controller for simultaneous frequency scaling and rate adaptation would have a nonlinear system model, we have designed PowerGPU, a two-tier coordinated RTR control architecture. The *outer loop* uses frequency scaling to control the average task RTR, while the *inner loop* adopts rate adaptation to control the individual RTR of each task on a smaller timescale.

## REFERENCES

- [1] Y. Wang *et al.*, “Gcaps: Gpu context-aware preemptive priority-based scheduling for real-time tasks,” in *ECRTS*, 2024.
- [2] S. K. Saha *et al.*, “Stgm:spatio-temporal gpu management for real-time tasks,” in *RTCSA*, 2019.
- [3] S. Subramanian *et al.*, “FC-GPU: Feedback Control GPU Scheduling for Real-time Embedded Systems,” in *ACM TECS*, 2025.
- [4] S. Subramanian, R. Joshi *et al.*, “SEEB-GPU: Early-Exit Aware Scheduling and Batching for Edge GPU Inference,” in *SEC*, 2025.
- [5] A. Zou *et al.*, “Rtgpu: Real-time gpu scheduling of hard deadline parallel tasks with fine-grain utilization,” *IEEE TPDS*, 2023.
- [6] G. C. Buttazzo *et al.*, “Elastic task model for adaptive rate control,” in *RTSS*, 1998.
- [7] J. Guo and C. Peng, “Adaptive tracking: Dynamic frame rate for drone-based target following,” in *UEMCON*, 2024.
- [8] C. Alippi *et al.*, “An adaptive sampling algorithm for effective energy management in wireless sensor networks with energy-hungry sensors,” *IEEE TIM*, 2009.
- [9] G. Chen *et al.*, “Enabling latency-guaranteed co-location of inference and training for reducing data center expenses,” in *ICDCS*, 2024.
- [10] Y. Wang *et al.*, “Balancing energy efficiency and real-time performance in gpu scheduling,” in *RTSS*. IEEE, 2021.
- [11] Y. Wang, K. Ma *et al.*, “Temperature-constrained power control for chip multiprocessors with online model estimation,” *ACM SIGARCH computer architecture news*, 2009.